

Introduction to Loops

A **loop** is used when we want to execute the same code multiple times — turning repetitive, unmanageable code into clean, scalable logic.

✗ Without a Loop

```
console.log("Welcome");  
console.log("Welcome");  
console.log("Welcome");  
console.log("Welcome");  
console.log("Welcome");
```

- Repetitive code
- Difficult to maintain
- Not scalable

✓ With a Loop

```
for(let i = 1; i <= 5; i++){  
  console.log("Welcome");  
}
```

The same task is completed in just a few lines — clean, readable, and scalable.

Real World Examples



Instagram Feed

Loads hundreds of posts automatically using loops.



Student Records

Processes thousands of student entries efficiently.



YouTube Comments

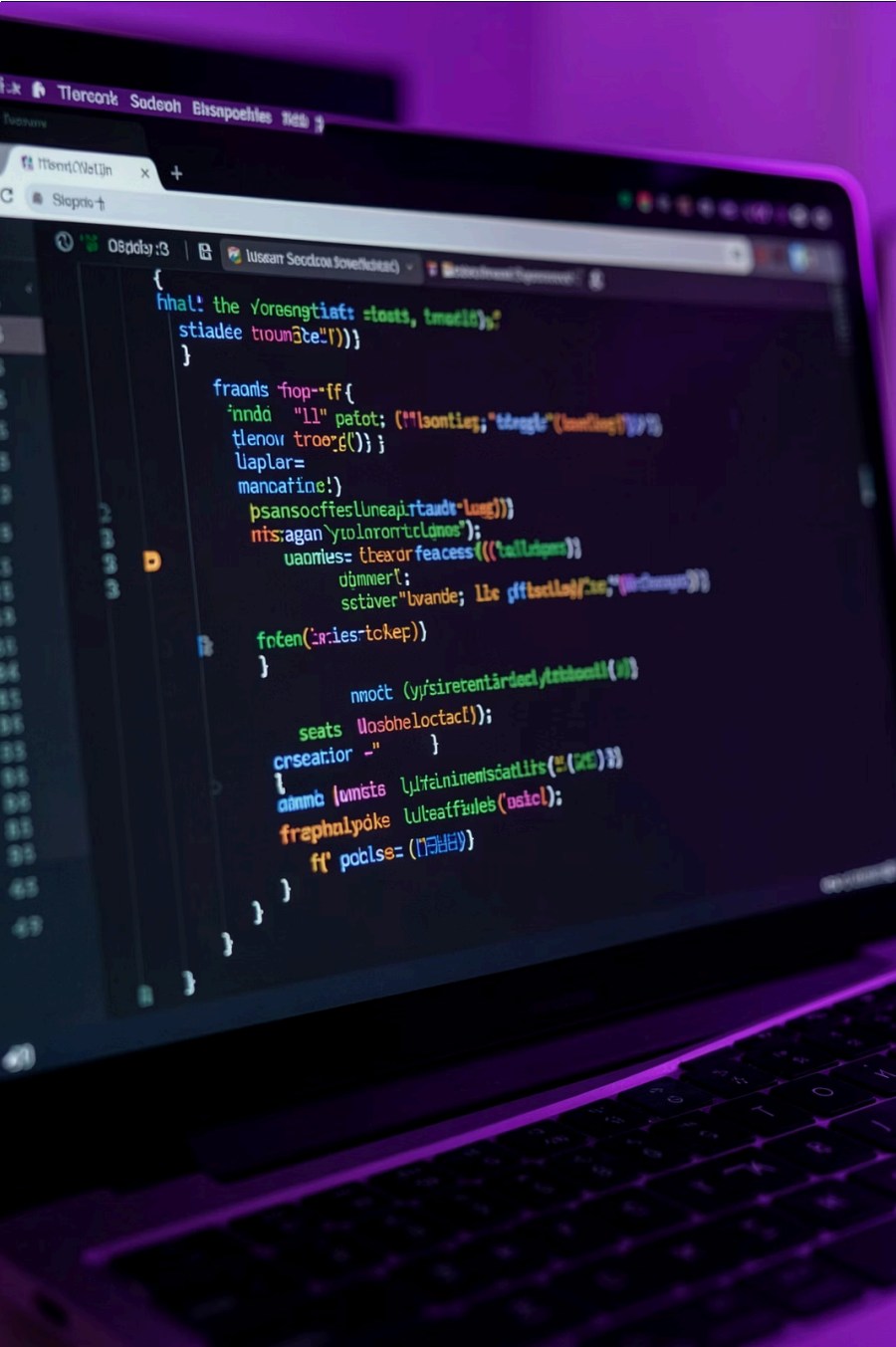
Displays many comments one after another dynamically.



E-Commerce Products

Displays product lists dynamically from a database.

🔗 **Classroom Question:** If I want to print my name 100 times, should I write 100 `console.log` statements? — Students will understand why loops are essential!



The for Loop

Use a `for` loop when you know **exactly** how many times the code should run.

Syntax

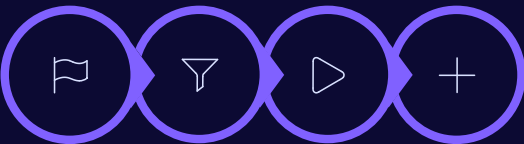
```
for(initialization; condition; increment){
  // code
}
```

Example

```
for(let i = 1; i <= 5; i++){
  console.log(i);
}
```

Output

```
1
2
3
4
5
```



Initialization Condition Execute Increment

1

Initialization
let i = 1 — Set the starting point of the loop.

2

Condition Check
i <= 5 — If true, execute the code block.

3

Execute
console.log(i) — Run the code inside the loop.

4

Increment
i++ — Increase the counter by 1 and repeat.

Interview Question: When should we use a for loop? **Answer:** When the number of iterations is known in advance.

Real World for Loop Examples

The for loop is incredibly versatile. Here are four practical examples demonstrating different use cases and increment patterns.

1

Print Student Names

```
for(let i = 1; i <= 10; i++){  
  console.log("Student", i);  
}
```

Used for attendance systems and student lists. Outputs Student 1 through Student 10.

2

Even Numbers

```
for(let i = 2; i <= 20; i += 2){  
  console.log(i);  
}
```

`i += 2` increases by 2 every iteration. Outputs: 2, 4, 6 ... 20.

3

Odd Numbers

```
for(let i = 1; i <= 20; i += 2){  
  console.log(i);  
}
```

Starting at 1 and incrementing by 2. Outputs: 1, 3, 5 ... 19.

4

Reverse Counting

```
for(let i = 10; i >= 1; i--){  
  console.log(i);  
}
```

Using `i--` to count down. Outputs: 10, 9, 8 ... 1.



Product Display

Loop through product arrays to render listings dynamically.



Report Generation

Iterate over data sets to generate formatted reports.



Record Processing

Process thousands of database records efficiently.



The **while** Loop

Use a while loop when you **don't know how many times** the loop should run — the loop continues as long as the condition remains true.

Syntax

```
while(condition){  
  // code  
}
```

Example

```
let i = 1;  
  
while(i <= 5){  
  console.log(i);  
  i++;  
}
```

Output

```
1  
2  
3  
4  
5
```

Real World Example: ATM PIN

```
while(enteredPIN !== correctPIN){  
  // ask again  
}
```

The user could enter:

- 1 attempt
- 2 attempts
- 10 attempts

We don't know beforehand — that's exactly when while is the right choice.



for vs while — Key Difference

for Loop

- Known iteration count
- Fixed number of runs
- Counter built into syntax

while Loop

- Unknown iteration count
- Dynamic number of runs
- Condition-driven execution

The `do...while` Loop

The `do...while` loop guarantees the code runs **at least once** before the condition is ever checked — unlike `while` which checks first.

Syntax

```
do{  
  // code  
} while(condition);
```

Example

```
let i = 1;  
  
do{  
  console.log(i);  
  i++;  
} while(i <= 5);
```

Output

```
1  
2  
3  
4  
5
```

Proof — Condition is False from the Start

```
let i = 10;  
  
do{  
  console.log("Executed");  
} while(i < 5);
```

Output

Executed

✓ Even though `i < 5` is **false**, the code still executes once. This is the defining behavior of `do...while`.

while Loop

```
while(condition){ }
```

Condition is checked **first**. If false from the start, the code never runs.



do...while Loop

```
do{ } while(condition);
```

Code is executed **first**. Condition is checked after the first run.

📄 🧑‍💻 **Real Example — Login Form:** A user must see the login page at least once before any validation occurs. `do...while` is the perfect fit for this scenario.

The `break` Statement

The `break` statement **stops the loop immediately** — no further iterations are executed once `break` is encountered.

Example

```
for(let i = 1; i <= 10; i++){  
  if(i === 5){  
    break;  
  }  
  
  console.log(i);  
}
```

Output

```
1  
2  
3  
4
```

The loop stops at `i = 5` — the number 5 is never printed because `break` fires before `console.log`.

Step-by-Step Execution

01

i = 1

Condition false → print 1

02

i = 2

Condition false → print 2

03

i = 3

Condition false → print 3

04

i = 4

Condition false → print 4

05

i = 5

`break` → loop ends immediately

Real World Uses of `break`



Search Operation

Found the product? Stop searching — no need to continue through the rest of the list.



Security Check

Unauthorized user detected? Stop execution immediately to prevent further access.



Validation

Correct input received? Stop the loop — no need to keep prompting the user.

The `continue` Statement

The `continue` statement **skips the current iteration** and moves on to the next one — the loop itself keeps running.

Example

```
for(let i = 1; i <= 5; i++){  
  if(i === 3){  
    continue;  
  }  
  
  console.log(i);  
}
```

Output

```
1  
2  
4  
5
```

Number 3 is skipped — the loop continues with `i = 4` after the `continue` fires.

Step-by-Step

- `i = 1 → print`
Condition false, execute normally.
- `i = 2 → print`
Condition false, execute normally.
- `i = 3 → continue (skip)`
Condition true — skip this iteration.
- `i = 4 → print`
Loop resumes normally.
- `i = 5 → print`
Loop completes normally.

break vs continue

`break`

```
break;
```

Stops the entire loop. No more iterations run after this point.

`continue`

```
continue;
```

Skips only one iteration. The loop continues with the next cycle.

📌 **Attendance Example:** `if(studentAbsent){ continue; }` — Skip the absent student and process all remaining students without stopping the loop.

Nested Loops

A **nested loop** is a loop placed inside another loop. The inner loop completes all its iterations for every single iteration of the outer loop.

Syntax & Example

```
for(let i = 1; i <= 3; i++){  
  for(let j = 1; j <= 3; j++){  
    console.log(i, j);  
  }  
}
```

Full Output

```
1 1 | 1 2 | 1 3  
2 1 | 2 2 | 2 3  
3 1 | 3 2 | 3 3
```

How It Executes

When i = 1

Inner loop runs fully: prints 1 1, 1 2, 1 3

When i = 2

Inner loop runs fully: prints 2 1, 2 2, 2 3

When i = 3

Inner loop runs fully: prints 3 1, 3 2, 3 3

Real Applications of Nested Loops



Multiplication Tables

Generate complete times tables by iterating rows and columns with nested loops.



Pattern Printing

Create star patterns, number pyramids, and other visual patterns in the console.



Matrix Operations

Traverse 2D arrays for mathematical matrix calculations and transformations.



Seating Arrangements

Map rows and columns of seats in theaters, classrooms, or stadiums.

Mini Task 1 — Multiplication Table Generator

Apply your knowledge of for loops to build a multiplication table generator — starting simple, then enhancing with template literals.

Basic Version

```
let number = 5;

for(let i = 1; i <= 10; i++){
  console.log(number * i);
}
```

Output

```
5
10
15
20
25
30
35
40
45
50
```

Enhanced Version (with Template Literals)

```
let number = 5;

for(let i = 1; i <= 10; i++){
  console.log(`${number} x ${i} = ${number * i}`);
}
```

Output

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
...
5 x 10 = 50
```

Concepts Learned in This Task



Loops

Using for to iterate a fixed number of times.



Multiplication

Applying arithmetic operators inside loop bodies.



Variables

Storing and reusing values like number and i.



Template Literals

Using backticks and \${} to format output strings dynamically.

Pattern Printing & Final Summary

Star Pattern with Nested Logic

```
for(let i = 1; i <= 5; i++){
  console.log("*".repeat(i));
}
```

Output

```
*
**
***
****
*****
```

How It Works

O1

Iteration 1

"*".repeat(1) → *

O2

Iteration 2

"*".repeat(2) → **

O3

Iteration 3-5

Pattern grows by one star each time.

✔ Complete Chapter Summary

Loop Types • for loop • while loop • do...while loop	Control Flow • break statement • continue statement • Nested loops	Practical Skills • Multiplication tables • Pattern printing • Real-world use cases
---	---	---

🏠 Homework Challenges

1. Number Pattern Print: 1 / 12 / 123 / 1234 / 12345 using nested loops.	2. Reverse Star Pattern Print stars in decreasing order: ***** down to *.
3. Multiplication Table Generator Take user input and generate a complete formatted table.	4. Even Numbers 1-100 Print all even numbers from 1 to 100 using a loop.

👏 Students who complete this chapter will have a solid foundation in loop logic — one of the most fundamental and powerful concepts in all of programming!